ViT-Micro on Atalla — measured schedule, DMA, instruction mix
Team notes, April 2026

Summary. We report static and dynamic metrics for ViT-Micro (B=1, T=32, D=32) from `run_graph.py` validate plus the functional simulator: schedule sparsity, SDMA bytes, retired-instruction mix by mnemonic class, and per-layer cosines vs. BF16 PyTorch. *Chained* means tensors move through Python's `activation_cache` so each emulated op sees the previous emulator output. Attention is `atalla_sdpa` with `ATALLA_SDPA_FLASH=1`: the fused flash kernel is compiled and run on the functional sim—there is no NumPy or PyTorch attention substitute in this path. NumPy short-circuits the emulator for `gelu` (vector-lane limit) and, with the default `ATALLA_FFN_ADD_NUMPY=1`, for the 2048-wide FFN bias `add` after the hidden projection; set `ATALLA_FFN_ADD_NUMPY=0` to force the AtallaC add image for that layer (known mismatch vs. ref on the current sim for that shape). Residual and other 1024-element adds use the C kernel. We tag claims as Tier 1 (simulator counts), Tier 2 (bytes + SP0/SP1 naming for GEMM CSV columns), or off-paper sketches—Tier 3 tiling art is out of this PDF. The main empirical punch is the scalar-heavy retired-instruction mix (Table 1) plus brutal static schedule sparsity (Tables 2–3).

Pipeline (how to read the tables). PyTorch FX is lowered in `run_graph.py`; supported ops emit C, compile through AtallaC → assembly → `build_compiler.compile_asm` (`schedule_program` packs VLIW rows), and the functional simulator runs the `.in` image. Regenerate the artifacts and this PDF with (from `atalla-graph/`): `ATALLA_SDPA_FLASH=1 python3 run_graph.py --model vit_micro --mode validate --validate-inputs chained --metrics-json out/graph/vit_micro_metrics.json --layer-metrics-csv out/graph/vit_micro_layer_metrics.csv` then `python3 docs/vit_micro/generate_writeup_figures.py` and `pdflatex` as in the header comment. Metrics land in `out/graph/vit_micro_metrics.json` and `out/graph/vit_micro_l` after validate.

- **Tier 1:** packet/slot stats from the static image; dynamic packets + retired ops from the sim; SDMA bytes; FLOP model; per-layer cosines on emulated nodes.
- **Tier 2:** CSV `reuse_R_A_gemm`/`reuse_R_W_gemm` only mean A/W if the emitted kernel matches that bank story—check each `.c`.
- Table 2 last row and per-node $\mathrm{AI}_{\mathrm{ld}}$ are model ratios (FLOPs / SDMA bytes), not hardware roofline AI.

Findings

Retired instructions (Table 1). Scalar ALU + scalar GMEM dominate dynamic retirements; GEMM/systolic is a few percent—so today's compiled kernels burn most committed ISA slots on scalar bookkeeping and scalar memory, not the MAC array. Buckets come from `misc/instruction_retire.py` (mnemonic classes), not a PMU.

Table 1: Measured. Dynamic retired instruction mix (all emulated kernels). One count per non-`nop.s` op; buckets follow `misc/instruction_retire.py` (mnemonic-based; not hardware-classified). Scalar ALU plus scalar GMEM dominate this run; GEMM/systolic is a small fraction.

Class	Count	% of retired ops
GEMM / systolic	2,720	2.27%
Vector ALU (.vv/.vi/.vs)	10,209	8.53%
Vector mem (vreg.ld/st)	15,380	12.85%
SDMA (scpad)	585	0.49%
Move / convert	1,568	1.31%
Scalar ALU (other)	53,209	44.45%
Scalar GMEM (lw/sw)	28,660	23.94%
Branch / control	7,369	6.16%
Total (retired ops)	**119,700**	100.00%

Static schedule (Tables 2, 3). Most static packet rows are empty of real ops before NOP fill; on rows with ≥ 1 real op, mean slot fill is still ∼27% (Table 2)—order one live op per four-slot row. At most one

vector + one memory op per packet, and control takes whole rows, so "heavy" co-issue in a mixed row has a rough 50% structural ceiling; observed ∼27% is already below that, so not all slack is "the ISA's fault." We don't split forced vs. schedulable gap here. Table 3 is the histogram of real-op counts per static row.

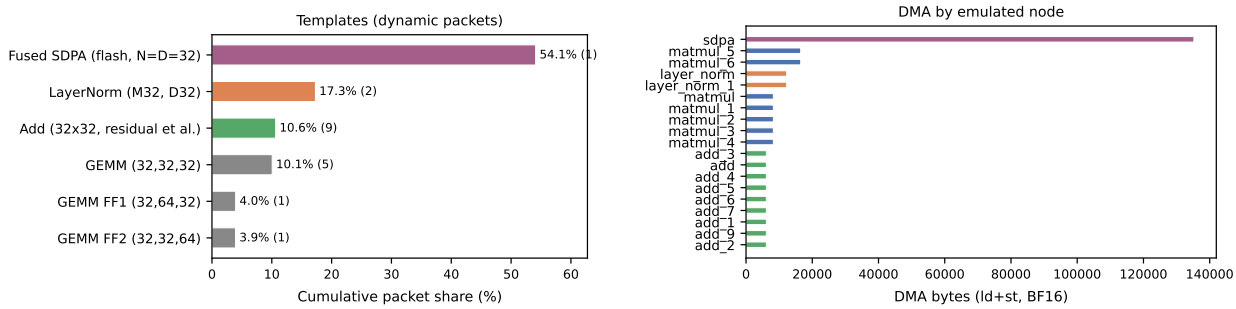
Table 2: Static instruction image (aggregated over emulated kernels). A packet row is four slots. **Empty rows** have no real op before `nop.s` fill. Last row: ISA-weighted FLOPs / SDMA BF16 bytes—*not* hardware roofline AI.

Metric	Value
Total static packet rows	4,250
Rows with ≥ 1 real instruction	4,250 (100.0%)
Empty rows	**0 (0.0%)**
Slot fill (all rows)	27.92%
Slot fill (non-empty rows only)	27.92%
Dynamic slot fill (retired / dyn. packets)	28.05%
Total FLOPs (model)	1,082,053
DMA bytes loaded	246,400
SP0 / SP1 loads	191,104 / 55,296
DMA bytes written	43,008
SP0 / SP1 stores	24,576 / 18,432
FLOPs / DMA (ld+st), model	3.739

Table 3: Histogram: how many real instructions occupy each static packet row (summed across kernels).

# real ops in row	Count	Share
1	3,832	90.16%
2	347	8.16%
3	64	1.51%
4	7	0.16%

Templates, DMA, retire mix (Figures 1, 2). Fig. 1a: cumulative dynamic packet share by template (fused SDPA ∼54% of dynamic packets in a typical flash run; LayerNorm and 32×32×32 GEMMs split most of the remainder). Fig. 1b: SDMA bytes per node (attention and FFN GEMMs dominate at this scale). Fig. 2: same buckets as Table 1 per template; LN leans vector-ALU vs. GEMM shapes, in line with higher η_{st} on LN rows in Table 4.



(a) Dynamic packet share by template.

(b) DMA bytes (loads + stores) per node.

Figure 1: Measured concentration (Tier 1).

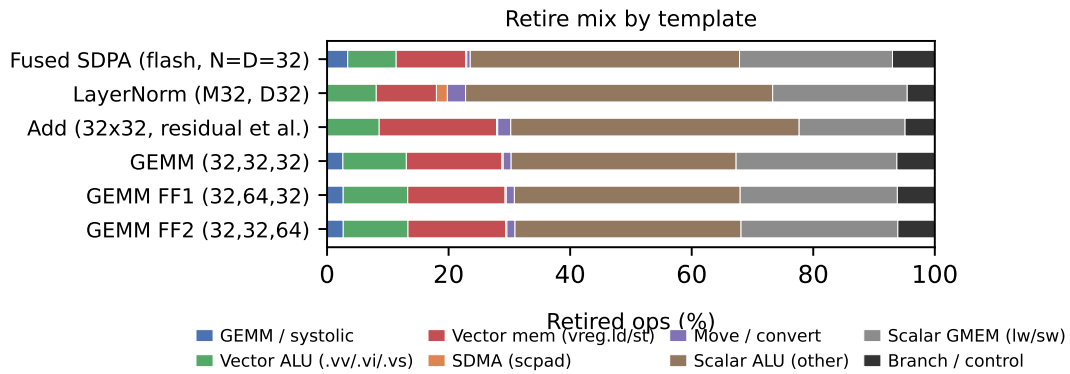


Figure 2: Measured. Retired-op mix by template.

Per-node (Table 4). Per-op cosines vs. BF16 ref are high for most emulated nodes; the chained end-to-end cosine is dominated by attention and the wide FFN stages (GELU + optional NumPy bias add) and is ∼0.98 in a typical `ATALLA_SDPA_FLASH=1` run with the default NumPy GELU and FFN add paths (see validate console `cos=`)—not a tight "paper" match without tightening those paths. η_{st} is ∼28% on this run; LayerNorm blocks differ from pure GEMM templates in vector vs. scalar retire mix (Table 2), not from higher MAC throughput.

Table 4: Per-node metrics (emulator layers): Pkt% = share of dynamic packets; η_{st} = static slot fill (non-`nop.s` / all static slots); AI_{ld} = model FLOPs / SDMA bytes loaded—not hardware AI.[1]

Name	Op	Map	Pkt%	P_{stat}	η_{st}	Bytes	FLOPs	AI_{ld}	cos
matmul	matmul	$32 \times 32 \times 32$	2.01	319	27.3%	8256	74326	11.97	1.00000
add_3	add	32×32	1.18	116	28.2%	6144	4470	1.09	1.00000
add	add	32×32	1.18	116	28.2%	6144	4470	1.09	1.00000
layer_norm	layernorm	LN 32×32	8.65	319	29.2%	12288	31336	3.06	0.99999
matmul_1	matmul	$32 \times 32 \times 32$	2.01	319	27.3%	8256	74326	11.97	0.99999
add_4	add	32×32	1.18	116	28.2%	6144	4470	1.09	0.99999
matmul_2	matmul	$32 \times 32 \times 32$	2.01	319	27.3%	8256	74326	11.97	0.99999
add_5	add	32×32	1.18	116	28.2%	6144	4470	1.09	0.99999
matmul_3	matmul	$32 \times 32 \times 32$	2.01	319	27.3%	8256	74326	11.97	0.99999
add_6	add	32×32	1.18	116	28.2%	6144	4470	1.09	0.99999
sdpa	atalla_sdpa	—	54.11	335	28.9%	135232	310248	2.33	0.25027
matmul_4	matmul	$32 \times 32 \times 32$	2.01	319	27.3%	8256	74326	11.97	0.44980
add_7	add	32×32	1.18	116	28.2%	6144	4470	1.09	0.67970
add_1	add	32×32	1.18	116	28.2%	6144	4470	1.09	0.98122
layer_norm_1	layernorm	LN 32×32	8.65	319	29.2%	12288	31336	3.06	0.98049
matmul_5	matmul	$32 \times 64 \times 32$	3.95	319	27.3%	16512	148638	11.97	0.97919
matmul_6	matmul	$32 \times 32 \times 64$	3.94	319	27.3%	16512	148635	11.97	0.98182
add_9	add	32×32	1.18	116	28.2%	6144	4470	1.09	0.98380
add_2	add	32×32	1.18	116	28.2%	6144	4470	1.09	0.98164

Scope and next steps

- **Scale:** T=D=32 is still toy vs. real ViT shapes—don't extrapolate without re-running.
- **Compiler:** scalar retirements + scalar GMEM (Table 1); slot slack vs. structure (Table 2).

[1] Tier 2: CSV `reuse_R_A_gemm`/`reuse_R_W_gemm` map SP0/SP1 loads to A/W by emitter convention only; audit each `.c` before reuse claims.

- **Validation:** bring GELU and/or the 2048-wide FFN add fully on-chip for stricter chained E2E; flash SDPA already runs on the sim. Audit SP0/SP1 per GEMM before trusting `reuse_R_*_gemm`.
- **Instrumentation:** split non-empty-row waste; optional Tier 2 pass over emitted `.c`.
- Tier 3 tiling slides live elsewhere unless tied to new counters.